Project Report

Differential Drive Robot Targeting a Waypoint and a Pose
- Using Pure Pursuit Controller
And Inverse Kinematic Model

Course: ECE 544 - 002, Winter 2023

By: Nayan Patel
UoM ID: 27900573

Table of Contents

1 Purpose 3
2 Scope 3
3 Overview 3
3.1 Project Objective 3
3.2 Background 3
3.3 Project Team 3
4 System Description 4
4.1 Robot Pictures 4
4.2 Robot HW Schematic 5
4.3 Functional System Overview 7
5 Software Architecture and Implementation 9
5.1 SW Architecture 9
5.2 SW Implementation 11
6 Testing 22
6.1 Test Set Up and Procedure 22
6.2 Results and Observations 23
7 Problems Faced and Solved 24
8 Areas of Further Improvement 25
9 Technical and Source Code References 25

1 Purpose

The document is the project report for the academic project that is part of course ECE 544, Winter 2023 term at University of Michigan-Dearborn.

2 Scope

This project report provides the technical details of the project titled *"Differential Drive Robot Targeting a Waypoint and a Pose – Using Pure Pursuit Controller and Inverse Kinematic Model".* It covers the overview of the project, system description, SW architecture and design, challenges faced during the project and references.

3 Overview

3.1 Project Objective

The main goal of the project is to start the differential drive robot from any arbitrary position on X-Y plane with a starting pose, and drive to a desired 2D waypoint (x, y) and stop there with a desired pose (θ). Implementing the complete "See-Think-Act" cycle on a differential drive robot and making it realized on real hardware is the ultimate objective of this exercise.

3.2 Background

Three Degrees of Freedom (DoF), allow the differential drive robots a great flexibility to move easily in linear motion, and to steer to any direction. Due to the use of only 2 robots (sometimes, one castor in front), the construction of the robot is pretty simple and control is less complex. And hence, they have a variety of applications, ranging from floor cleaning to commercial usage for loading and unloading materials.

Once a robot has the area map, for every move, ideally there are two target values to achieve: (1) A goal position on X-Y plane, AND (2) A target pose angle. There are several different algorithms available to drive the robot to its desired position and the pose. A pure pursuit algorithm is a closed loop controller algorithm which is simple use and when tuned up properly and combined with inverse kinematics model can generate precise wheel speed for each of the wheels. The pure pursuit model needs accurate measurement (or calculated values) of the absolute orientation of robot (i.e., position in X-Y plane and current pose).

Learning the pure pursuit algorithm with kinematics models in Matlab is the motivation behind this project. How to realizing simulation of the robot model with pure pursuit controller and kinematics models in a real physical robot system is the goal of the project.

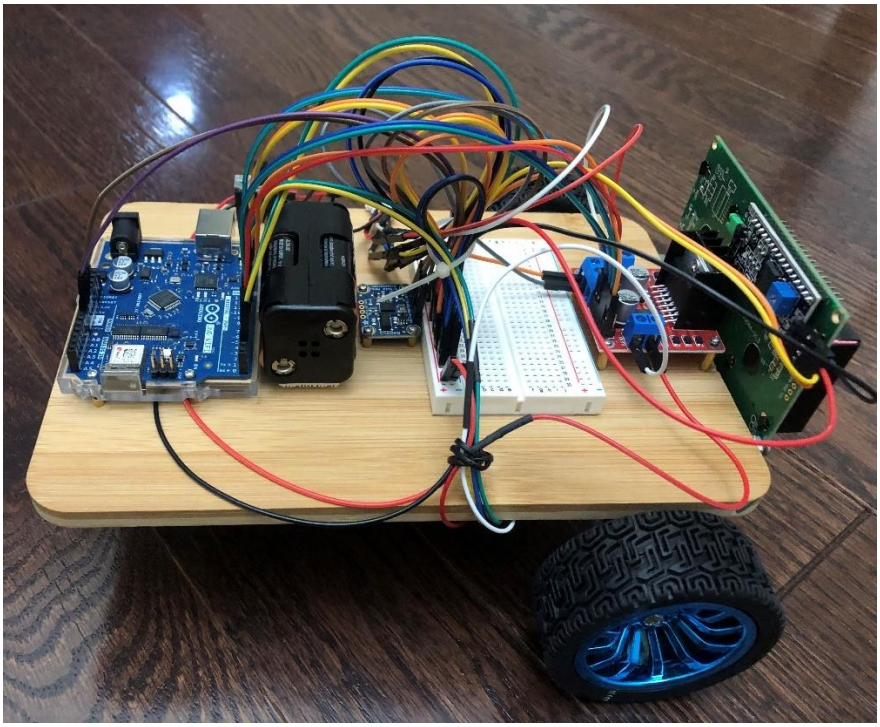
NOTE: There are several Arduino and other open-source code from different sources used to develop the SW for this project, mainly to develop the Driver SW and HW Abstraction Layer. Required changes were made in the original code and the code was adapted to the need of this project. The application software (pure pursuit controller, inverse kinematics model and position calculations etc.) has been developed based on referenced material. The reference to all those sources has been listed in Section 9

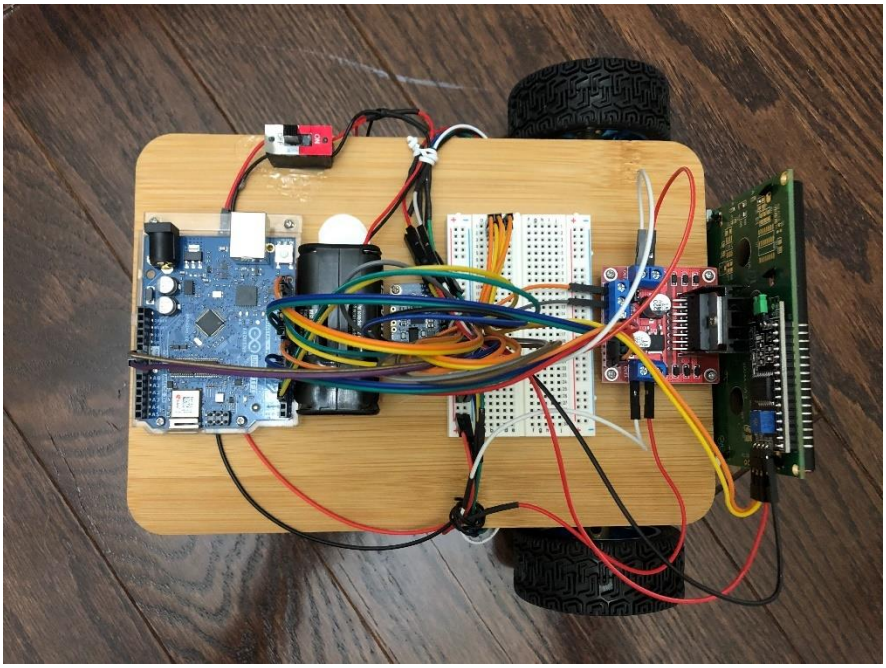
3.3 Project Team

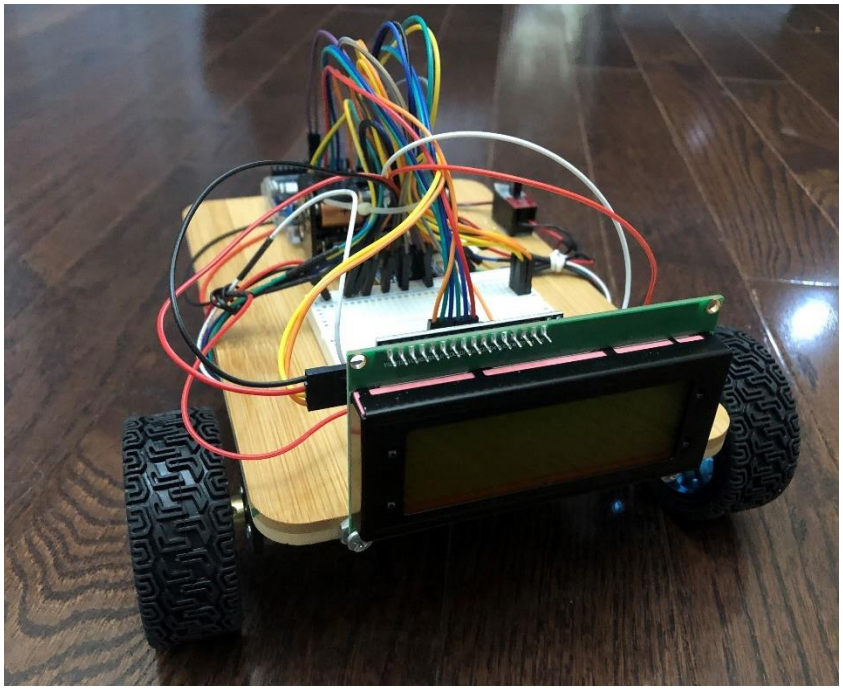
The author of this project report has individually and independently worked on this project and there is no one else in the team.

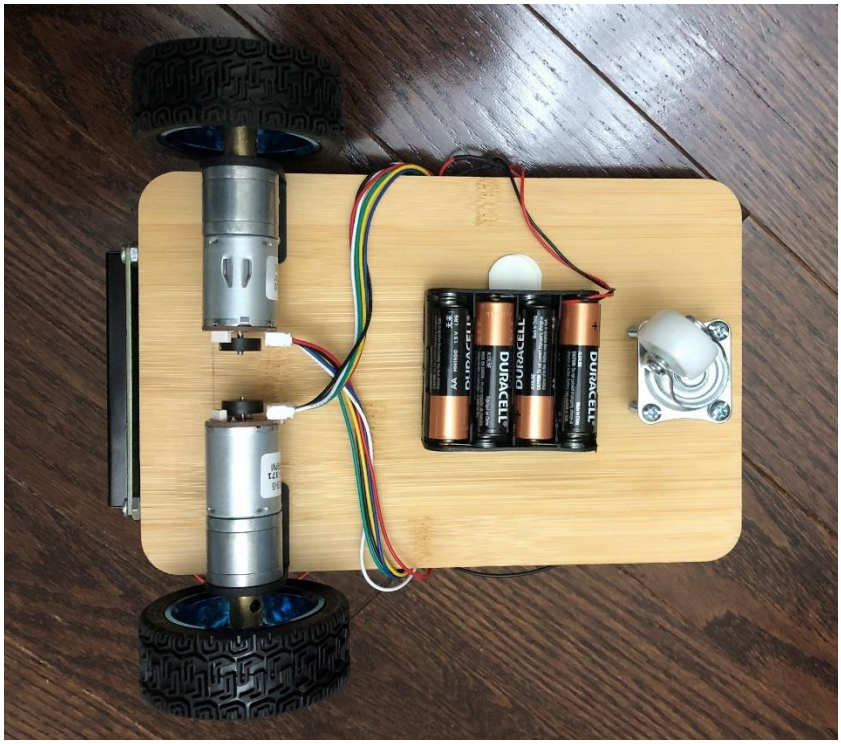
4 System Description

4.1 Robot Pictures



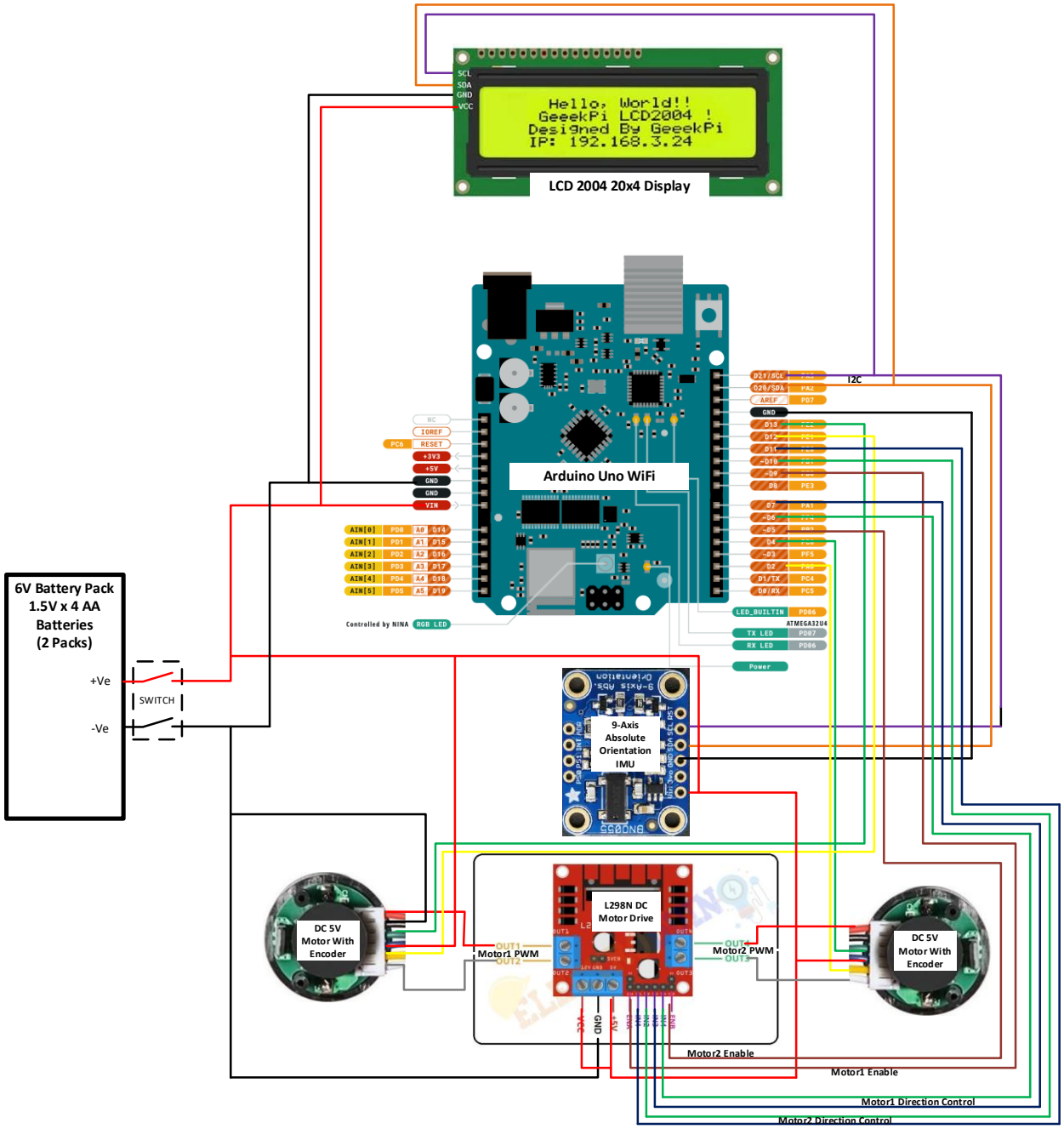






4.2 Robot HW Schematic

Following diagram shows the main hardware components that were used in this project:



List of HW components:

- Arduino UNO WiFi Microcontroller Board (1 No.)
- L298N Motor Control Drive (1 No.)
- BNO055 9-Axis Absolute Orientation IMU Sensor (1 No.)
- DC Motor with Magnetic Rotary Encoder (2 Nos.)
- Rubber Wheels (2 Nos.)
- 2004 20 x 4 LCD Display (1 No.)
- 6V DC Power Bank (1.5V x 4 AA Batteries) (2 No.)
- Breadboard (1 No.)
- Dual Pole Switch (1 No.)

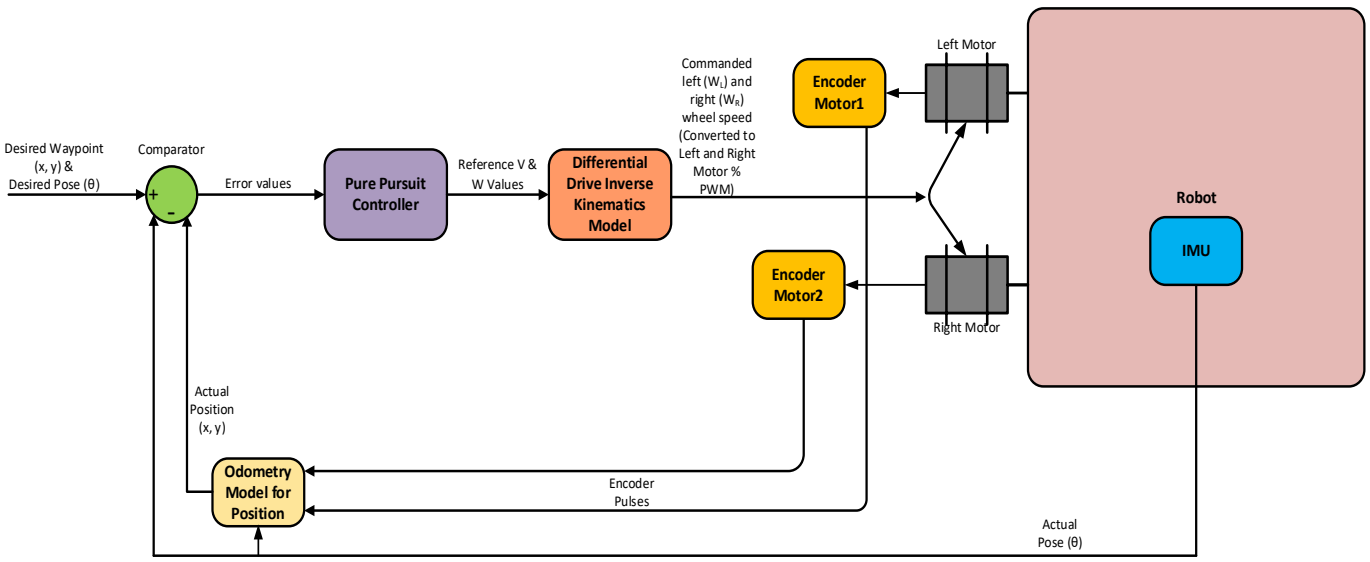
- Wooden Board (for Robot Chassis) (1 No.)

4.3 Functional System Overview

The differential drive robot system in this project mainly has following functional subsystems:

1) Pure Pursuit Controller
2) Inverse kinematics model
3) Sensors for angular orientation and angular movement measurement
4) Actuator to drive the wheels of the robot

Following diagram depicts the systems context:



Each of the subsystems is explained in detail as follows:

1) <u>Pure Pursuit Controller</u>

In this differential drive robotic system, there are three process values that are required to be controlled. A Pure Pursuit Controller algorithm is used as it can convert the error between desired a waypoint and a pose and current values of robot position and pose angle.

The block's input, parameter and output variables are as follows:

Inputs:

- Desired Waypoint & Pose: Target values of Linear X-position (X) (in m), Linear Y-position (Y) (in m) and angular position (θ) (in rad)
- Current Position & Pose: Current values of Linear X-position (X) (in m), Linear Y-position (Y) (in m) and angular position (θ) (in rad)

Outputs:

- Linear Velocity (V) (in m/s) and Angular Velocity (W) (in rad/s)

Following are equations for the Pure Pursuit Algorithm model that's used in this project:

$\rho = \sqrt{(\Delta x^2 + \Delta y^2)}$
$\alpha = -\theta + \text{atan2}(\Delta y / \Delta x)$
$\beta = -\theta - \alpha + \text{Goal Pose Angle}$

$$v = k_\rho * \rho$$
$$w = k_\alpha * \alpha + k_\beta * \beta$$

Where,
Δx = distance between desired and current x coordinate
Δy = distance between desired and current y coordinate
θ = current pose angle
v = Linear Velocity
w = Angular Velocity
k_ρ, k_α & k_β = Controller Gain (Tune Up) Values

The Lookahead Distance (LD) is to tune the response of the controller by setting the distance of how closely the robot follows the waypoints and how smoothly it moves along the traversal path. A robotic vehicle with higher LD produces smoother path, however takes broader turns at corners. Conversely, the vehicle with smaller LD moves closer to the waypoints and takes sharper turns and hence, it's prone to oscillations.

The current value of robot position (x, y) has been derived using following formula:

Linear displacement of each wheel: $\Delta U_{L/R} = \dfrac{\pi * \text{Wheel Diameter} * \text{Encoder Pulse Increment in a Cycle}}{Gear\ Ratio\ of\ the\ Motor * Encorder\ Resolution}$

Incremental Linear Displacement of Robot's CenterPoint: $\Delta U = \dfrac{\Delta U_{Left} + \Delta U_{Right}}{2}$

The absolute position of robot's CenterPoint on X-Y plane is:

$$X_i = X_{i-1} + \Delta U * \cos(\theta)$$
$$Y_i = Y_{i-1} + \Delta U * \sin(\theta)$$

Where θ is the relative angular position calculated using absolute angular position read from 9-Axis Absolute Accelerometer (i.e., BNO055 IMU Sensor).

The Pure Pursuit Controller is implemented on Arduino UNO WiFi Rev2 microcontroller system.

2) <u>Inverse kinematics model</u>

The "Differential Drive Inverse Kinematics" block has following I/O and uses Inverse Kinematics Equations as stated below.

Inputs:
- Reference Linear Velocity (V) (in m/s) and Angular Velocity (W) (in rad/s)

Outputs
- Left and right wheel speeds ω_L and ω_R, in rad/s

Inverse Kinematics Equations:

$$\omega_L = \frac{1}{R}\left(v - \frac{\omega L}{2}\right), \quad \omega_R = \frac{1}{R}\left(v + \frac{\omega L}{2}\right)$$

The Inverse Kinematics Model is implemented on Arduino UNO WiFi Rev2 microcontroller system.

3) <u>Sensors for angular orientation and angular movement measurement</u>

Following Sensors are used in this robot system:

1. <u>Magnetic Rotary Encoder:</u> Each of the two motors used in the robot has a magnetic rotary encoder. This encoder is generating 11 pulses per one rotation of the motor. However, there is a gear attached between the motor and the wheel shaft for speed reduction (gear ratio of 47.5:1) and hence, for one rotation of the wheel, total pulses generated by encoder is 11 * 47.5 = 522.5
2. <u>Absolute Orientation IMU:</u> Arduino BNO055 9-Axix Absolute Orientation IMU is used in the system for providing absolute value of angular position in refence to x-axis.

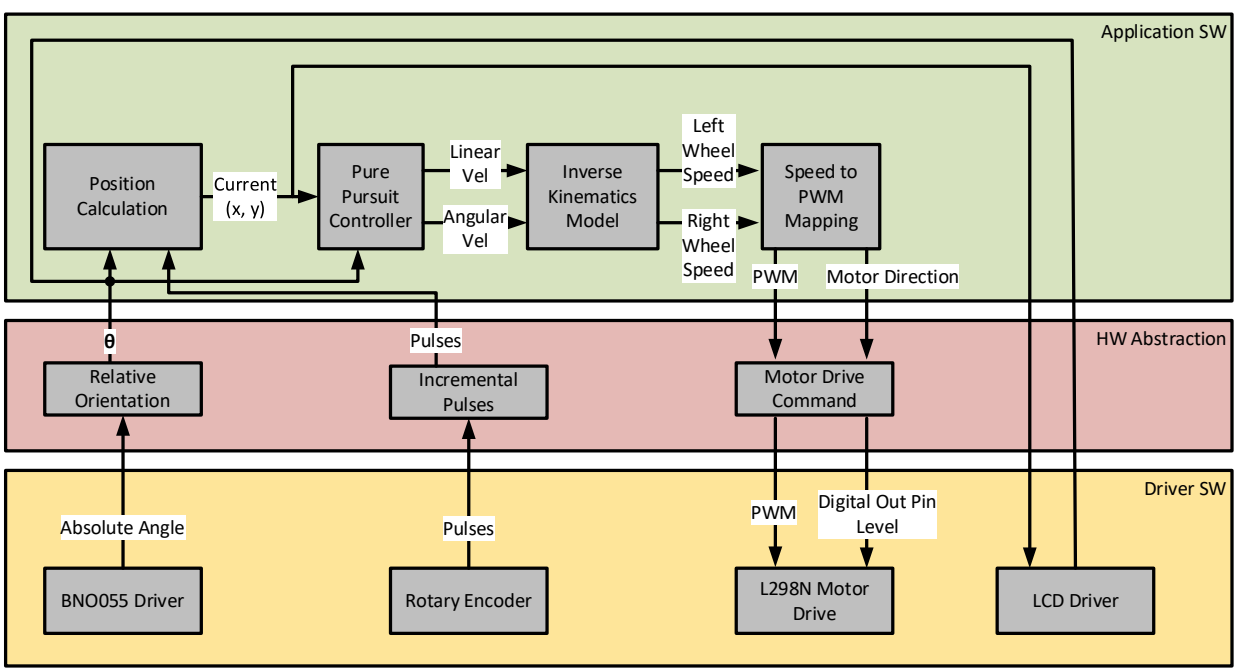
4) <u>Actuator to drive the wheels of the robot</u>

There are two 6V DC motors used to drive the differential wheels. One castor wheel is attached in the front that provides two degree of maneuverability total: Linear and Angular.

5 Software Architecture and Implementation

5.1 SW Architecture

The layered SW architecture is designed in this project. The below diagram shows that functional SW architecture:



1. **<u>Driver SW:</u>** The driver SW layer contains the following set of drivers:

iii. BNO055: The control and communication of BNO055 drive is over I2C bus. The I2C address used is 0x28. The Arduino I2C driver library and data abstraction code is used as base for creating custom APIs to read orientation data.

iv. Rotary Encoder: The pulses of each encoder is measured using interrupt driven Digital Input pins. This type of encoder generates 522.5 pulses in one complete rotation of the attached wheel and hence, an interrupt is absolutely necessary to get accurate number of pulses and to avoid any error in estimating the linear displacement.

v. L289N Motor Driver: The L298N Motor Driver requires two main inputs: % PWM Duty Cycle and Digital Input for motor direction. Each of the two motors, can be independently controlled but using one L298N only to update both motors at the same time.

iv. LCD Driver: The 2004 20x4 LCD display communicates to the Arduino UNO WiFi controller over I2C bus. The I2C address of LCD display is 0x27.

2. **HW Abstraction Layer:** The HW abstraction layer contains following function blocks:

   i. Relative Orientation: This extracts the relative orientation of the robot position. It reads the absolute angle value from the BNO055 driver and converts to relative orientation for the robot, the angle relative to the X-axis.

   ii. Incremental Pulses: The encoder pulses read from the driver are converted to incremental pulses in specific cycle. Also, due to the fact that both motors are mounted back-to-back facing drive shaft opposite, the sign of pulses reading from one motor will need to be reversed, so it matches with other motor.

   iii. Motor Drive Command: The calculated value of PMW from application layer for each motor will be transferred to driver using right API. The same APIs is being used to set the direction of the motor.

3. **Application SW Layer:** The major components in Application SW Layer are as follows:

   i. Position Calculation: Based on the current orientation (angular position) of the robot and encoder pulses, the current position of the robot is calculated.

   ii. Pure Pursuit Controller: The pure pursuit controller gets input from Position Calculation component and calculates the linear and angular velocity.

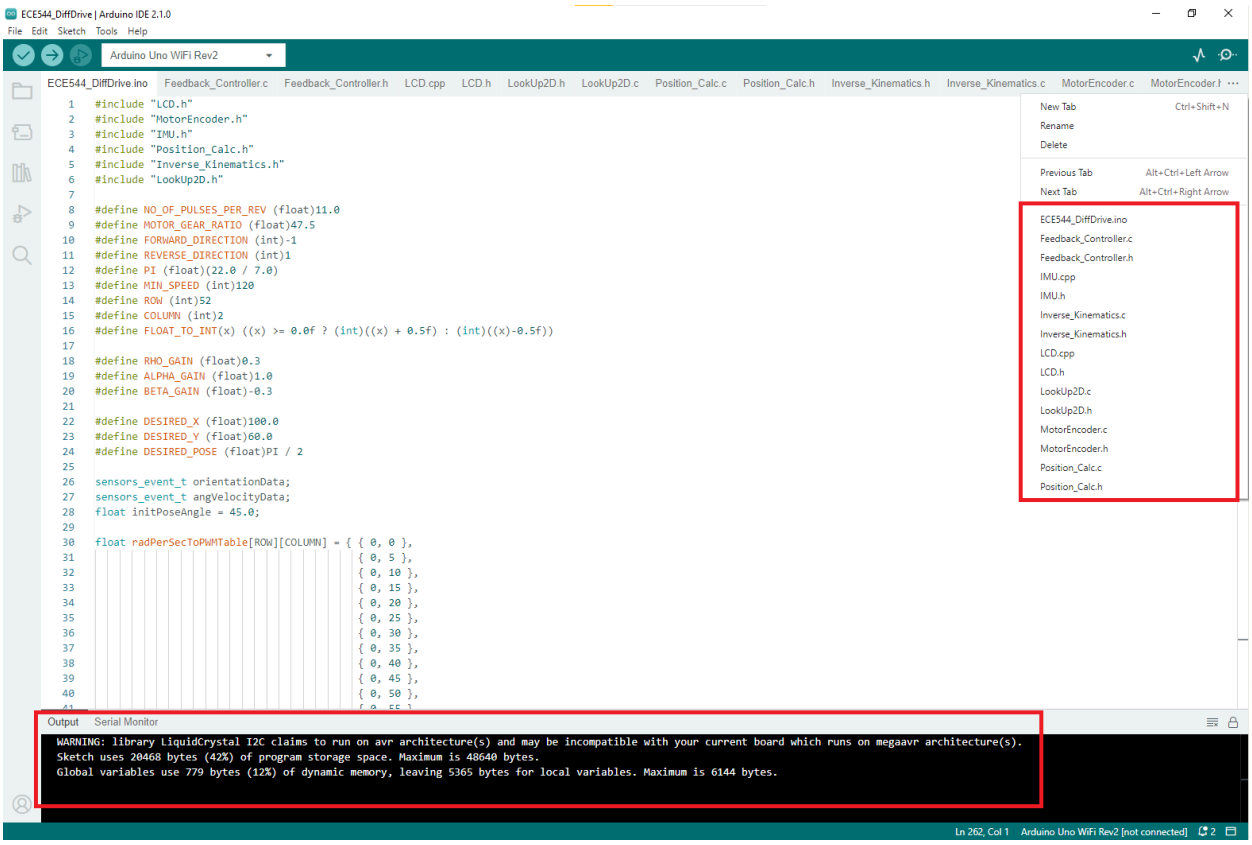
   iii. Inverse Kinematics Model: The inverse kinematics model receives the desired values of linear and angular velocity values. It converts those to left and right wheel speed values.

iv. <u>Speed to PWM Mapping:</u> The DC motor used in this project has a non-linear relationship between input speed and output PWM. The Wheel Speed vs. look up table has been used with interpolation/extrapolation algorithm to calculate the PWM command for the motors.

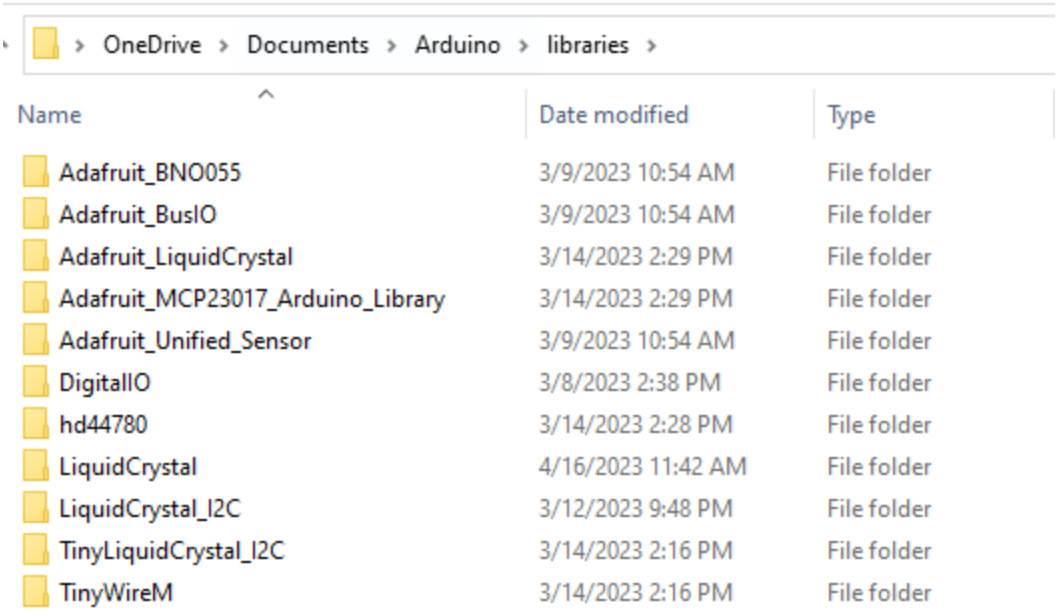
5.2 SW Implementation

The entire SW for this robot was implemented using Arudino IDE v2.1.0. The code was written in mixed of C++/C flavors based on the need and basis of the open-source code (wherever used).

The code files that are implemented in this project are shown in the following picture. It also shows the successful code build status along with the memory usage data.



Several open-source Arduino and open-source libraries that have been used in this project. They are listed below:



Code snippets for important SW blocks are as follows:

- <u>BNO055 Driver</u>

```
#define BNO055_I2C_ID              (int8_t)55
#define BNO055_I2C_ADDRESS         (uint8_t)0x28

Adafruit_BNO055 bno = Adafruit_BNO055(BNO055_I2C_ID, BNO055_I2C_ADDRESS, &Wire);

void imuInit(void){

  /* Initialise the sensor */
  if(!bno.begin())
  {
    /* There was a problem detecting the BNO055 ... check your connections */
    Serial.print("Ooops, no BNO055 detected ... Check your wiring or I2C ADDR!");
    while(1);
  }

  delay(10);

  bno.setExtCrystalUse(true);
}

sensors_event_t imuOrientationData(void){

```

```
  /* Get a new sensor event */
  sensors_event_t orientationData;
  bno.getEvent(&orientationData, Adafruit_BNO055::VECTOR_EULER);

  return orientationData;
}

sensors_event_t imuAngVelocityData(void){

  /* Get a new sensor event */
  sensors_event_t angVelocityData;
  bno.getEvent(&angVelocityData, Adafruit_BNO055::VECTOR_GYROSCOPE);

  return angVelocityData;
}
```

- <u>Rotary Encoder:</u>

```
static int posCountEn1 = 0;
static int posCountEn2 = 0;

extern void motorEncoderInit(void){

  pinMode(ENCA_M1, INPUT);  // sets the Encoder_output_A pin as the input
  pinMode(ENCB_M1, INPUT);  // sets the Encoder_output_B pin as the input

  attachInterrupt(digitalPinToInterrupt(ENCA_M1), readEncoderEn1, RISING);
  attachInterrupt(digitalPinToInterrupt(ENCA_M2), readEncoderEn2, RISING);
}

extern void readEncoderEn1(void) {

  int b = digitalRead(ENCB_M1);

  if (b > 0) {
    posCountEn1++;
  }
  else {
    posCountEn1--;
  }
}
extern void readEncoderEn2(void) {

  int b = digitalRead(ENCB_M2);

```

```
  if (b > 0) {
    posCountEn2++;
  }
  else {
    posCountEn2--;
  }
}

extern int getPosCountEn1(void) {

  return -posCountEn1;
}

extern int getPosCountEn2(void) {

  return posCountEn2;
}
```

- <u>L289N Motor Driver:</u>

```
extern void motorEncoderInit(void){

  pinMode(ENCA_M1, INPUT);  // sets the Encoder_output_A pin as the input
  pinMode(ENCB_M1, INPUT);  // sets the Encoder_output_B pin as the input
  pinMode(OUT1_M1, OUTPUT);
  pinMode(OUT2_M1, OUTPUT);
  pinMode(PWM_OUT_M1, OUTPUT);

  pinMode(ENCA_M2, INPUT);  // sets the Encoder_output_A pin as the input
  pinMode(ENCB_M2, INPUT);  // sets the Encoder_output_B pin as the input
  pinMode(OUT1_M2, OUTPUT);
  pinMode(OUT2_M2, OUTPUT);
  pinMode(PWM_OUT_M2, OUTPUT);
}

extern void setMotor(int dir, int pwmVal, int pwm, int out1, int out2) {
  analogWrite(pwm, pwmVal);

  if (dir == 1) {
    digitalWrite(out1, HIGH);
    digitalWrite(out2, LOW);
  } else if (dir == -1) {
    digitalWrite(out1, LOW);
    digitalWrite(out2, HIGH);
  }
  else {
```

```
    digitalWrite(out1, LOW);
    digitalWrite(out2, LOW);
  }
}
```

- <u>LCD Driver:</u>

```
LiquidCrystal_I2C lcd(0x27, 20, 4);

void lcdInit(void){

  digitalWrite(SDA, 1);
  digitalWrite(SCL, 1);

  // lcd.begin();
  lcd.init();
  lcd.backlight();
  // lcd.clear();
  lcd.setCursor (0,0); //
  lcd.print("Robojax LCD2004 Test");

  delay(10);
}

void printLCDScreen(lcdData data){

  char X[5], Y[5], Theta1[5];

  lcd.clear();

  lcd.setCursor (0, 0); //character zero, line 0
  lcd.print("Set Val"); // print text
  lcd.setCursor (8, 0); //character 4, line 0
  lcd.print("|Actual Val"); // print text
  delay(0.01);

  lcd.setCursor (0, 1); //character zero, line 0
  lcd.print("Xs:"); // print text
  lcd.setCursor (4, 1); //character 4, line 0
  dtostrf(data.robotTargetPos_X, 3, 1, X);
  lcd.print(X); // print text
  lcd.setCursor (8, 1); //character zero, line 0
  lcd.print("|Xa:"); // print text
  lcd.setCursor (13, 1); //character 14, line 0
  dtostrf(data.robotPos_X, 3, 1, X);
  lcd.print(X); // print text
```

```
  delay(0.01);

  lcd.setCursor (0, 2); //character zero, line 1
  lcd.print("Ys:"); // print text
  lcd.setCursor (4, 2); //character 4, line 1
  dtostrf(data.robotTargetPos_Y, 3, 1, Y);
  lcd.print(Y); // print text
  lcd.setCursor (8, 2); //character zero, line 1
  lcd.print("|Ya:"); // print text
  lcd.setCursor (13, 2); //character 14, line 1
  dtostrf(data.robotPos_Y, 3, 1, Y);
  lcd.print(Y); // print text
  delay(0.01);

  lcd.setCursor (0, 3); //character zero, line 2
  lcd.print("Ts:"); // print text
  lcd.setCursor (4, 3); //character 4, line 0
  dtostrf(data.imuTargetOrientiation_X, 4, 1, Theta1);
  lcd.print(Theta1); // print text
  lcd.setCursor (8, 3); //character zero, line 2
  lcd.print("|Ta:"); // print text
  lcd.setCursor (13, 3); //character 4, line 0
  dtostrf(data.imuOrientiation_X, 4, 1, Theta1);
  lcd.print(Theta1); // print text
  delay(0.01);
}
```

- <u>Relative Orientation:</u>

```
  currentOrientationTheta = orientationData.orientation.x;

  if (currentOrientationTheta == 0) {
    relativeOrientation = currentOrientationTheta;
  } else if (currentOrientationTheta <= 360.00 &&
currentOrientationTheta >= 180) {
    relativeOrientation = 360 - currentOrientationTheta;
  } else {
    relativeOrientation = 0 - currentOrientationTheta;
  }
```

- <u>Incremental Pulses:</u>

```
extern int getPosCountEn1(void) {
```

```
  return -posCountEn1;
}

extern int getPosCountEn2(void) {

  return posCountEn2;
}
  /* Motor 1 encoder output reading */
  currPosCountsM1 = getPosCountEn1();
  noOfRevM1 = (float)currPosCountsM1 / (NO_OF_PULSES_PER_REV *
MOTOR_GEAR_RATIO);
  pulseCountIncrM1 = currPosCountsM1 - prevPosCountM1;
  Serial.print("Pulse this cycle M1 = ");
  Serial.println(pulseCountIncrM1);

  /* Motor 2 encoder output reading */
  currPosCountsM2 = getPosCountEn2();
  noOfRevM2 = (float)currPosCountsM2 / (NO_OF_PULSES_PER_REV *
MOTOR_GEAR_RATIO);
  pulseCountIncrM2 = currPosCountsM2 - prevPosCountM2;
  Serial.print("Pulse this cycle M1 = ");
 Serial.println(pulseCountIncrM2);
```

- <u>Motor Drive Command:</u>

```
/* Set motor direction based on the wheel speed value */
if (whlSpd.wheelSpeedLeft < 0) {
    setDirectionMotorLeft = REVERSE_DIRECTION;
  }
  if (whlSpd.wheelSpeedRight < 0) {
    setDirectionMotorRight = REVERSE_DIRECTION;
  }

/* Stop robot within certain distance once reached goal position and pose
This is to avoid unncessary oscillations and unstable behavior */
  if ((currentPos.pos_x <= DESIRED_X + 2 && currentPos.pos_x >= DESIRED_X
- 2) &&
      (currentPos.pos_y <= DESIRED_Y + 2 && currentPos.pos_x >= DESIRED_Y
- 2) &&
      (currentPos.pos_x <= DESIRED_POSE + PI / 10 && currentPos.pos_x >=
DESIRED_POSE - PI / 10)) {
    setMotor(setDirectionMotorLeft, 0, PWM_OUT_M1, OUT1_M1, OUT2_M1);
    setMotor(setDirectionMotorRight, 0, PWM_OUT_M2, OUT1_M2, OUT2_M2);
  }else{
    setMotor(setDirectionMotorLeft, leftMotorPWM, PWM_OUT_M1, OUT1_M1,
OUT2_M1);
```

```
    setMotor(setDirectionMotorRight, rightMotorPWM, PWM_OUT_M2, OUT1_M2, OUT2_M2);
```

- <u>Position Calculation:</u>

```
#define PI (float)(22.0 / 7.0)
#define WHEEL_DIAMETER (float)65.0  // in mm
#define WHEEL_BASE (float)185.0     // in mm
#define NO_OF_PULSES_PER_REV (float)11.0
#define MOTOR_GEAR_RATIO (float)47.5

static float odometryConversionFactor = 0.0;  // Conversion factor that translates encoder pulses into linear wheel displacement
static float degToRadConversionFactor = 0.0;

extern void robotPositionInit(void) {
  odometryConversionFactor = (PI * WHEEL_DIAMETER) / (MOTOR_GEAR_RATIO * NO_OF_PULSES_PER_REV);
  degToRadConversionFactor = PI / 180.0;
}

extern robotPosition getRobotPosition(float prevPosX, float prevPosY, int motor1PulseCount, int motor2PulseCount, float orientationTheta) {

  robotPosition currentPos;
  float incTravelDistanceWheelLeft = 0.0;   // Incremental distance traveled by each wheel, left
  float incTravelDistanceWheelRight = 0.0;  // Incremental distance traveled by each wheel, right
  float incLinearDisplacement = 0.0;        // Incremental Linear Displacement of the robot's center point C
  float currentOrientationInRad = 0.0;
  float incOrientation = 0.0;
  float relativeOrientation = 0.0;
  static float prevRelativeOrientation;

  incTravelDistanceWheelLeft = odometryConversionFactor * (float)motor1PulseCount;
  incTravelDistanceWheelRight = odometryConversionFactor * (float)motor2PulseCount;

  incLinearDisplacement = (incTravelDistanceWheelLeft + incTravelDistanceWheelRight) * 0.5;

  incLinearDisplacement = incLinearDisplacement / 10;
```

```

  // incOrientation = (incTravelDistanceWheelRight - incTravelDistanceWheelLeft) / WHEEL_BASE;

  // relativeOrientation = prevRelativeOrientation + incOrientation;

  relativeOrientation = (360 - orientationTheta) * degToRadConversionFactor;

  currentPos.pos_x = (prevPosX + incLinearDisplacement * cos(relativeOrientation));
  currentPos.pos_y = (prevPosY + incLinearDisplacement * sin(relativeOrientation));

  prevRelativeOrientation = orientationTheta;

  return currentPos;
}
```

- <u>Pure Pursuit Controller:</u>

```
extern refVelocity calcRefVelocity(currentRobotPosition currentVals, targetRobotPosition desiredVals, feedbackCntrlrTuneUpVals tuneUpVals){

  float rho, alpha, beta;
  refVelocity refVel;

  /* Calculate the polar coordinates */
  rho = sqrt(pow((desiredVals.goalPos_x - currentVals.currPos_x), 2) + pow((desiredVals.goalPos_y - currentVals.currPos_y), 2));
  alpha = -currentVals.currOri_theta + atan2((desiredVals.goalPos_y - currentVals.currPos_y), (desiredVals.goalPos_x - currentVals.currPos_x));
  beta = -currentVals.currOri_theta - alpha + desiredVals.goalOri_theta;

  /* Calculating the linear and angular velocity */
  refVel.LinVel = tuneUpVals.k_rho * rho;
  refVel.AngVel = tuneUpVals.k_alpha * alpha + tuneUpVals.k_beta * beta;
  refVel.rho = rho;
  refVel.alpha = alpha;
  refVel.beta = beta;

  return refVel;
}
```

- <u>Inverse Kinematics Model:</u>

```
#define WHEEL_BASE            (float)185.0 // in mm
```

```
#define WHEEL_RADIUS          (float)(65.0/2.0) // in mm

extern wheelSpeedCommand calWheelSpeed(refVelocity refVelVals){
  wheelSpeedCommand whlSpd = {0};

  whlSpd.wheelSpeedLeft = (1/WHEEL_RADIUS) * (refVelVals.LinVel -
((refVelVals.AngVel * WHEEL_BASE) * 0.5));
  whlSpd.wheelSpeedRight = (1/WHEEL_RADIUS) * (refVelVals.LinVel +
((refVelVals.AngVel * WHEEL_BASE) * 0.5));

  return whlSpd;
}
```

- <u>Speed to PWM Mapping</u>

```
float radPerSecToPWMTable[ROW][COLUMN] = { { 0, 0 },
                                            { 0, 5 },
                                            { 0, 10 },
                                            { 0, 15 },
                                            { 0, 20 },
                                            { 0, 25 },
                                            { 0, 30 },
                                            { 0, 35 },
                                            { 0, 40 },
                                            { 0, 45 },
                                            { 0, 50 },
                                            { 0, 55 },
                                            { 0, 60 },
                                            { 0, 65 },
                                            { 0.001047198, 70 },
                                            { 0.001047198, 75 },
                                            { 0.001047198, 80 },
                                            { 0.001047198, 85 },
                                            { 0.001047198, 90 },
                                            { 0.00523599, 95 },
                                            { 0.01570797, 100 },
                                            { 0.161268492, 105 },
                                            { 0.082728642, 110 },
                                            { 0.122522166, 115 },
                                            { 0.165457284, 120 },
                                            { 0.215722788, 125 },
                                            { 0.269129886, 130 },
                                            { 0.326725776, 135 },
                                            { 0.385368864, 140 },
                                            { 0.446106348, 145 },
                                            { 0.506843832, 150 },
                                            { 0.57072291, 155 },
```

```
                                                { 0.635649186, 160 },
                                                { 0.700575462, 165 },
                                                { 0.767596134, 170 },
                                                { 0.831475212, 175 },
                                                { 0.89535429, 180 },
                                                { 0.95818617, 185 },
                                                { 1.018923654, 190 },
                                                { 1.080708336, 195 },
                                                { 1.143540216, 200 },
                                                { 1.2042777, 205 },
                                                { 1.263967986, 210 },
                                                { 1.32470547, 215 },
                                                { 1.38230136, 220 },
                                                { 1.440944448, 225 },
                                                { 1.49749314, 230 },
                                                { 1.554041832, 235 },
                                                { 1.608496128, 240 },
                                                { 1.661903226, 245 },
                                                { 1.714263126, 250 },
                                                { 1.766623026, 255 } };

float **arr = (float **)malloc(ROW * sizeof(float *));

for (int i = 0; i < ROW; i++) {
  arr[i] = (float *)malloc(COLUMN * sizeof(float));
}

for (int i = 0; i < ROW; i++) {
  for (int j = 0; j < COLUMN; j++) {
    arr[i][j] = radPerSecToPWMTable[i][j];
  }
}

leftMotorPWM = interpolation2D(arr, ROW, COLUMN,
fabs(whlSpd.wheelSpeedLeft));
rightMotorPWM = interpolation2D(arr, ROW, COLUMN,
fabs(whlSpd.wheelSpeedRight));
```

```
extern float interpolation2D(float **radPerSecToPWMTable, int row, int col,
float RPSInputVal) {

  float retVal = 0.0;
  float distance = 0.0;

  /* If the input is lower than first value or higher than last value,
extropolate */
```

```
  if (RPSInputVal <= radPerSecToPWMTable[0][0]) {
    retVal = radPerSecToPWMTable[0][1];
  } else if (RPSInputVal >= radPerSecToPWMTable[row - 1][0]) {
    retVal = radPerSecToPWMTable[row - 1][1];
  } else {
    /* Search from first element to second last element */
    for (int i = 1; i < row - 1; i++) {

      if (RPSInputVal < radPerSecToPWMTable[i][0] && RPSInputVal >=
radPerSecToPWMTable[i - 1][0]) {
        if (RPSInputVal == radPerSecToPWMTable[i - 1][0]) {
          retVal = radPerSecToPWMTable[i - 1][1];
        } else {
          distance = (RPSInputVal - radPerSecToPWMTable[i - 1][0]) /
(radPerSecToPWMTable[i][0] - radPerSecToPWMTable[i - 1][0]);
          retVal = distance * (radPerSecToPWMTable[i][1] -
radPerSecToPWMTable[i - 1][1]) + radPerSecToPWMTable[i - 1][1];
        }
      }
      else{
        // do nothing
      }
    }
  }
  return retVal;
}
```

6 Testing

6.1 Test Set Up and Procedure

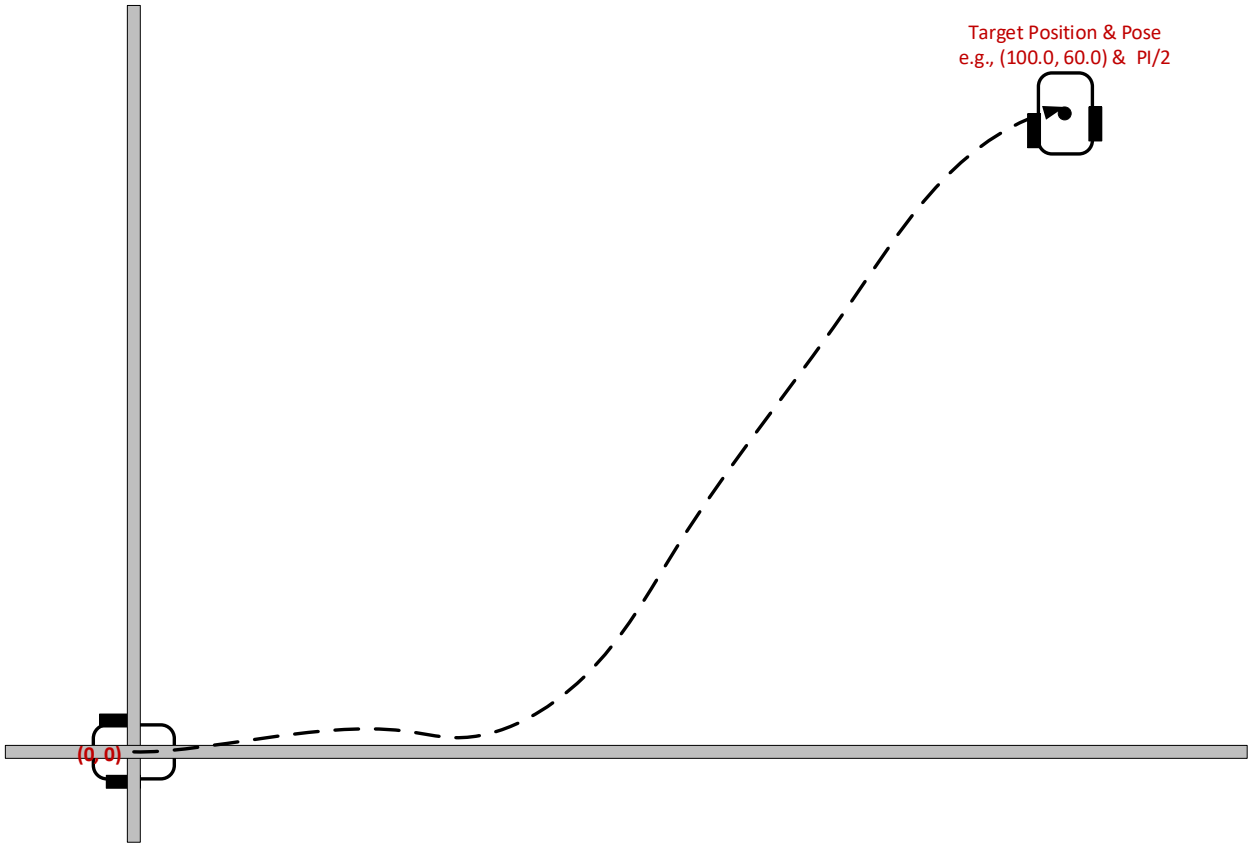
Following test steps are performed to verify the robot behavior and validate against the end goal:

1. The desired position (x, y) and pose (θ) (e.g., 100.0, 60.0 and PI/2) are entered in the code. Also, the controller tune-up values (e.g., 0.3, 0.8 & -0.5) are changed.
2. The program is flashed via Arduino JTAG over USB.
3. The robot is then placed on a hardwood surface where the first quadrant of the X-Y plane is created using painter's tape. The axis are arbitrary and does not align with actual longitude and latitude.
4. The robot's starting point is (0, 0) with initial pose of 0 rad.
5. The 6V supply to the system is turned on.
6. After a small pause, the controller will start running and the robot moves towards desired goal.
7. The trajectory of the robot is observed by logging the LCD display for the current position (x, y) and pose (θ).
8. The robot's behavior is observed for following:
    - Oscillations during linear and angular movement

- Unstable behavior e.g., robot spins around, at the start or on the way to goal point
- The trajectory robot follows
- Speed with what robot moves and turns angle
- Any unintended reset/restart of the system

6.2 Results and Observations

After tuning the controller gains with several runs, the robot was able achieve the target position: The desired result was not 100% perfect but it still is within acceptable limit.



Following observations were noted:

- The robot does not stop completely at the end point. It even becomes unstable and starts rotating in a circle at a variable speed which is a known issue with Pure Pursuit controller algorithm.
- The pure pursuit model cannot achieve the goal point and pose in a straight trajectory between start and end points.
- Small oscillations even during a linear travel are imminent due to the system constraints but can be minimized by fine tuning of the gains.
- Increasing rho gain helps with the increase in linear speed but during the turning, it creates big overshoot and sometimes spin of the robot.

- Increasing alpha and beta gain values increases the angular velocity command, but it causes one wheel to rotate at a faster rate than the other during turns and it causes de-stabilization of robot during turns, so a slowest possible speed during turns is safer for the robot.
- The behavior of the robot is not repeatable

7 Problems Faced and Solved

During the course of this entire project, there were several problems that were faced.

1. Problem: When calculated the angular position by integrating the wheel acceleration value from Arduino's in-built IMU over time, an erroneous value was observed which wasn't usable at all. That was due to the noise in the sensor output.

   Solution: Added BNO055 9-Axis Absolute Accelerometer that directly provides angular positions and it's accurate enough for this application.

2. Problem: The I2C communication between Arduino UNO WiFi and the BNO055 did not work at first.

   Solution: Several attempts were made to debug the issue. It was found that the I2C address of BNO055 was incorrect. A special Arduino program was created to read the default address of the IMU and that was used in the code to successfully fetch the IMU data.

3. Problem: The gear ratio of the DC motor wasn't available from the supplier. Without gear ration, no. of revolution cannot be calculated and hence, no linear wheel displacement will be known.

   Solution: Reverse engineering was done to find out the exact gear ratio. The motor was rotated for one exact revolution and the pulse counts received from encoder was used in following formula to find the gear ratio:

   Gear Ratio = Current Pulse Counts from Encoder / No. of Pulses In One Revolution

4. Problem: The DC motor that's used in this project is a very low rpm (max 100 RPM) motor. It doesn't run below certain threshold of % PWM DC and also, it has a non-linear relationship with the speed. Hence, the commanded wheel speed cannot be mapped directly to desired % PWM DC needed to drive the motor.

   Solution: The motor was run at all the % PWM DC ranging from min of 0% to max allowed 255% and the calculated speed in RPM was logged. The RPM was converted to Rad/Sec and a lookup table of Rad/Sec vs. % PWM DC was created that provided a right amount of % PWM DC based on required speed command.

5. Problem: The Arduino controller was continuously resetting during the testing.

   Solution: It was debugged and found that the 1 bank of 6V DC power was not sufficient to provide enough power, specifically during switching the motor direction. Due to the momentary dip in the voltage, the controller was resetting continuously. A second bank of 6V DC power was added in parallel and the problem was not seen again.

6. Problem: The robot does not stop completely at the end point. It even becomes unstable and starts rotating in a circle at a variable speed which is a known issue with Pure Pursuit controller algorithm.

   Solution: The PWM % DC is set to 0 when robot reaches within a smaller radius of the desired waypoint and within a few degrees of desired pose angle.

8 Areas of Further Improvement

Following areas have been identified that can improve the performance of the robot and also, code execution:

- An additional PID controller can be used to control the wheel speed. That can continuously monitor and correct the speed which can help to avoid oscillations, specifically around starting, direction switching and end points.
- An even better DC motor with more precise PWM resolution can be used for finer control of speed.
- A second LCD screen can be added to display some debug data. Keep the computer connected via USB cable while the robot is running is somewhat impractical.
- A loop with fixed amount of cycle time may bring better and repeatable performance.
- More powerful battery with several hours of run time would bring more consistent motor performance.

9 Technical and Source Code References

Following sources have been used for technical contents and software code:

1. Experiment 1 & 2 of ECE 544 (Mobile Robots) course at University of Michigan-Dearborn, Winter 2023 term.
2. Book "Autonomous Mobile Robots" by Ronald Siegwart & Illah R. Nourbakhsh
3. https://www.mathworks.com/help/nav/ug/pure-pursuit-controller.html
4. https://www.mathworks.com/discovery/inverse-kinematics.html?s_tid=srchtitle_inverse%20kinematics_1
5. http://www-personal.umich.edu/~johannb/Papers/pos96rep.pdf
6. https://howtomechatronics.com/tutorials/arduino/arduino-dc-motor-control-tutorial-l298n-pwm-h-bridge/
7. https://www.electroniclinic.com/arduino-dc-motor-speed-control-with-encoder-arduino-dc-motor-encoder/
8. https://learn.adafruit.com/adafruit-bno055-absolute-orientation-sensor/arduino-code
9. https://www.arduino.cc/reference/en/libraries/